Particle Physics Simulation

Debug report: defects, surprises and corrections from the build

Olajide Badejo

August 2026

Abstract

This is the companion to the main report and it ranks equally with it. The main report says what the engine does; this one says what went wrong on the way and what each failure taught, because in a project whose output is numbers the history of the numbers that were wrong is worth as much as the ones that were right.

Entries are grouped by theme. Environment covers the four toolchain defects that had to be cleared before a single kernel compiled, one of which would have silently produced a binary for the wrong GPU architecture. Performance covers the two results that contradicted my expectations: fusion delivering +0.1 percent where I expected ten to twenty, and shared memory tiling turning out not to be a bandwidth optimisation at all. Physics and tooling covers the defects in the correctness apparatus itself, including two cases where the test rather than the code was wrong, and a data corruption bug that put a random seed into a tile width column.

The most valuable entries here are the ones where nothing crashed. A build targeting the wrong architecture, a benchmark reading a corrupted column and an optimisation that quietly does nothing all produce plausible output, and none of them announces itself.

Contents

1 Environment **2**

1.1 nvcc 13.3 refuses GCC 16 as a host compiler 2

1.2 CUDA 13 removed clockRate and memoryClockRate 2

1.3 CMake could not identify the CUDA compiler at all 3

1.4 The build silently targeted the wrong GPU architecture 3

2 Performance **5**

2.1 Kernel fusion, expected ten to twenty percent, measured +0.1 percent 5

2.2 Shared memory tiling is not a bandwidth optimisation on this problem 6

2.3 The specified tile width is not the fastest 6

2.4 Absolute timings depend on the thermal state of the card 7

3 Physics and tooling **8**

3.1 The hash collision that would have counted neighbours twice 8

3.2 A diagnostic that would have perturbed the run it measured 9

3.3 A test that failed for a reason that was entirely my fault 9

3.4 A benchmark that recorded a random seed as a tile width 9

3.5 LaTeX macro names cannot contain digits 10

3.6 BibTeX put en dashes back after they had been removed 10

4 What the failures had in common **11**

4.1 The dangerous defects were the silent ones 11

4.2 A tolerance without a scale is not a tolerance 11

4.3 Size the optimisation before writing it 11

4.4 Verification has to be independent of the thing verified 12

4.5 The environment is part of the experiment 12

Chapter 1

Environment

Four defects stood between a clean checkout and a compiling kernel. None was interesting in itself; all four are recorded because three of them produce misleading rather than obvious failures.

1.1 nvcc 13.3 refuses GCC 16 as a host compiler

Symptom. Compiling a trivial CUDA file with `-ccbin g++-16`:

```
crt/host_config.h:137:2: error: #error -- unsupported GNU version! gcc
versions later than 15 are not supported!
```

Root cause. CUDA 13.3 supports GCC up to and including 15 and enforces that with a preprocessor check, so this is a deliberate refusal rather than a parse failure in some header. The machine has GCC 13, 14, 15 and 16 installed and defaults to 15, so the problem only appears if the specified floor of GCC 16.1 is taken literally, which it was.

Options. Pass `-allow-unsupported-compiler` and use GCC 16 anyway; drop to GCC 15.2, the newest supported host compiler; or install an older toolkit.

Fix and why. GCC 15.2. What the floor was really buying is the C++23 host standard, and GCC 15.2 provides that in full. Suppressing the toolkit's own compatibility check on a project whose entire output is numbers that are supposed to be trustworthy is the wrong trade: a miscompiled force kernel that still runs is the worst failure mode available here. Dropping the toolkit would have given up first class `sm_120` support on this card.

Verification. A kernel plus a CUB radix sort compiles, links and runs correctly under `-ccbin g++-15 -arch=sm_120 -std=c++23`. Recorded as a substitution in `PROGRESS.md`.

1.2 CUDA 13 removed clockRate and memoryClockRate

Symptom. The device probe would not compile:

```
error: class "cudaDeviceProp" has no member "clockRate"
error: class "cudaDeviceProp" has no member "memoryClockRate"
```

Root cause. Both fields were deprecated during the CUDA 12 series and removed in CUDA 13. The information is still available through `cudaDeviceGetAttribute`.

Fix and why. Query both through the attribute interface. This matters more than a compile fix normally would: the memory clock and bus width are what the theoretical peak bandwidth is computed from, and that figure is the denominator of every bandwidth statement in the main report. Hard coding it from a specification sheet would have meant quoting a number that had never been measured on the machine that produced the results, which is exactly what the no number without a run rule exists to prevent.

Verification. The probe reports 192 bits at the measured memory clock, giving 672.0 GB/s, which agrees with the manufacturer figure.

1.3 CMake could not identify the CUDA compiler at all

Symptom. Configuration failed inside `project()`, before any project code was reached:

```
/usr/include/c++/15/type_traits(3303): error: __bool_constant is not a template
Error limit reached.
100 errors detected in the compilation of "CMakeCUDACompilerId.cu".
```

The confusing part was that nvcc had just compiled my own test kernel against the same standard library without complaint.

Root cause. The difference was the standard flag. My test passed `-std=c++20` explicitly. nvcc 13.3 defaults to C++17, and the libstdc++ 15 headers it is paired with here use `if consteval` inside `bits/c++config.h`, which is C++23 syntax. Under the default standard the front end cannot parse its own standard library, so nvcc fails on an empty translation unit. Reduced to two lines the failure is unambiguous:

```
c++config.h(586): error: expected a "("
     if consteval { return true; } else { return false; }
```

CMake identifies the CUDA compiler by compiling a probe with `CMAKE_CUDA_FLAGS` and nothing else. `CMAKE_CUDA_STANDARD` has no effect there, because it applies to targets and no targets exist yet.

Fix and why. Seed `CMAKE_CUDA_FLAGS` with the standard before `project()`, guarded so a value passed on the command line still wins. Requiring every person who clones the repository to pass an extra flag would have broken the promise that a clean clone plus two make commands reproduces everything.

Verification. Configuration reports `The CUDA compiler identification is NVIDIA 13.3.73 with host compiler GNU 15.2.0`.

1.4 The build silently targeted the wrong GPU architecture

Symptom. None. Everything compiled and would have run. The only trace was a status line reading `CUDA architectures: 75`, which is Turing, not the Blackwell part in the machine.

Root cause. The default was being set with a guard placed after `project()`:

```
if(NOT DEFINED CMAKE_CUDA_ARCHITECTURES)
  set(CMAKE_CUDA_ARCHITECTURES 120)
endif()
```

Enabling the CUDA language inside `project()` assigns a default architecture immediately, so by the time the guard runs the variable is already defined and the branch never fires.

Fix and why. Move the guard above `project()` and make it a cache variable, which is the only position where it can observe the undefined state while still losing to a value passed on the command line.

Why this is the most important entry in the chapter. Nothing broke. The code would have run on the RTX 5070 through just in time compilation of the embedded PTX, correctly and at the wrong speed. Every timing number in the benchmark ladder would have been measured against a binary built for the wrong architecture, and the only symptom would have been results somewhat worse than expected, which is indistinguishable from the code simply being slow. A defect that produces plausible numbers is far more dangerous than one that crashes.

Verification. The status line now reads `CUDA architectures: 120`, and `cuobjdump --list-elf` on a built object reports `sm_120` SASS rather than PTX alone. The latter check is the real one: the status line reports intent, the object file reports fact.

Chapter 2

Performance

Two expectations in the original design were contradicted by measurement. The specification anticipated this and asked that the correction be written here rather than quietly absorbed, which is the right instruction: an expectation that was wrong and got corrected is more useful than one that was never written down.

2.1 Kernel fusion, expected ten to twenty percent, measured +0.1 percent

The expectation. Fusing the force evaluation with the Verlet update removes a round trip of the acceleration array, $12N$ bytes written and $12N$ bytes read, plus one kernel launch per step. Stated in bytes that sounds substantial, and ten to twenty percent is roughly right for a memory bound kernel.

The measurement. At 200,000 particles fusion is worth +0.1 percent. At 10,000 particles it is worth +2.1 percent. At some intermediate sizes it was slightly slower than the unfused path, within run to run variation.

Root cause of the wrong expectation. Arithmetic intensity. The all pairs sum performs $O(N^2)$ arithmetic on $O(N)$ data, so the ratio of operations to bytes grows without limit with the problem size. The saving can be bounded before any profiler is involved: $24N$ bytes at 200,000 particles is 0.0087 milliseconds at the measured streaming bandwidth of 549.9 GB/s, against a step of 45.78 milliseconds. That is 0.019 percent, and it is a ceiling rather than an estimate. No implementation of fusion, however good, could have delivered ten percent here.

Confirmation from counters. Nsight Compute puts device memory throughput at 0.55 percent of peak for the naive kernel and 0.45 percent for the tiled one. There was never any device memory pressure for fusion to relieve.

What I changed. Nothing in the code; the fused kernel is kept because it is the right choice at small particle counts and it halves the memory footprint of a step. What changed is the practice: an optimisation is now sized against the resource it claims to relieve, using the measured ceiling, before it is written. Two minutes of arithmetic would have predicted this result and saved the effort of being surprised by it.

2.2 Shared memory tiling is not a bandwidth optimisation on this problem

The expectation. Tiling is conventionally introduced as a way of reducing device memory traffic, and I introduced it that way. The prediction that follows is that the naive kernel is limited by memory bandwidth and the tiled kernel relieves it.

The measurement. Tiling delivers a factor of 1.93, which is real and large. But device memory throughput is 0.55 percent of peak before the change and 0.45 percent after. Device memory was never the constraint, so the conventional explanation of why tiling helps is wrong here even though the tiling itself is right.

What is actually happening. The counters point at the first level cache and the load store pipeline:

	throughput, percent of peak			per launch	
variant	L1 and load store	L2	device memory	global load sectors	warp issue
naive	80.19	1.77	0.55	1,250.0M	50.12
tiled	39.93	1.76	0.45	19.6M	74.53
fused	40.15	1.80	0.57	19.6M	74.94

Global load sectors fall by a factor of 63.8 and load store instructions by a factor of 3.8. First level cache throughput falls from 80.2 percent of peak, close to saturation, to 39.9 percent. The decisive number is warp issue efficiency, which rises from 50.1 percent to 74.5 percent: the naive kernel spends about half its time with nothing to issue because its warps are waiting on the load store pipe. Tiling moves the reads into shared memory and the pipe stops being the bottleneck.

Why this mattered practically. Had I trusted the textbook description and tried to improve the naive kernel by attacking device memory traffic, by compressing the state or adding prefetching, nothing at all would have happened. The profile pointed at the right resource; reasoning from the familiar description would not have.

Control. The arithmetic instruction counts are equal across all three variants to within a fraction of a percent, which is what licenses attributing the difference to data movement rather than to the variants computing different things. Those same counters also let the conventional twenty floating point operations per interaction be checked against the 18.0 actually retired.

2.3 The specified tile width is not the fastest

Symptom. A sweep of tile widths at one hundred thousand particles shows the specified width of thirty two is the slowest of the four measured.

	tiled		fused	
tile width	ms per step	TFLOP/s	ms per step	TFLOP/s
32	12.658	15.80	13.096	15.27
64	12.996	15.39	12.023	16.63
128	11.911	16.79	11.812	16.93
256	11.842	16.89	11.758	17.01

Root cause. Two barriers are paid per tile, so a narrower tile pays them more often per unit of arithmetic. A width of 256 runs at 11.842 milliseconds against 12.658, a difference of 6.4 percent.

Fix and why. The default stays at thirty two. The ladder exists to measure the design as specified, and changing the tile width partway would have made the rungs incomparable with the design they were meant to test. The sweep is reported instead, so the choice is informed rather than accidental, and the right resolution for future work is to expose the width as a configuration field.

2.4 Absolute timings depend on the thermal state of the card

Symptom. A tile width sweep run immediately after the full benchmark sweep produced timings about fifteen percent higher than the same sweep run from cold. Nothing in the code had changed.

Root cause. Sustained clocks fall as the card heats. This is ordinary behaviour, but it is large enough to swamp several of the effects being measured here, including the entire fusion result.

Fix and why. Two measures. Every comparison in the reports is made within a single session, so all points in a comparison share a thermal state. And each point keeps the fastest of three trials rather than the mean, because everything that perturbs a measurement makes it slower and never faster, so the minimum is the least contaminated estimate.

Verification. Across the hot and cold runs the relative ordering of the tile widths was identical and the percentage difference between the widest and narrowest tile was reproduced to within a tenth of a percentage point, even though the absolute milliseconds differed by fifteen percent. That is the evidence that the conclusions rest on relative differences and survive the variation.

Chapter 3

Physics and tooling

The entries here are about the correctness apparatus rather than the physics kernels. Two of them are cases where the test was wrong rather than the code, which is worth recording precisely because a wrong test is more dangerous than a wrong implementation: it either blocks correct work or, far worse, passes when it should not.

3.1 The hash collision that would have counted neighbours twice

The pathology. Spatial hashing is many to one by construction. Two distinct grid cells can map to the same bucket. The neighbour search visits the twenty seven cells around a particle and scans each one's bucket, so if two of those twenty seven cells collide in the table, the particles in that bucket are scanned twice and counted twice.

The reason this is nasty is that the usual defence does not work. Filtering candidates by distance removes particles from unrelated cells, because they are genuinely far away. It does nothing about double counting, because those particles really are within the smoothing radius; they are simply being added to the sum more than once. The density would come out high by an amount that depends on the hash function rather than on the fluid, and the fluid would still look like a fluid.

Fix and why. Each particle stores an exact cell identity packed into 64 bits as three 21 bit fields, and the bucket scan compares it before accepting a candidate. This rejects both classes at once, at the cost of one 64 bit compare per candidate. The alternative of deduplicating the twenty seven hashed buckets before scanning would cost hundreds of comparisons per particle rather than one per candidate.

Verification. This is the one defect in the project that was designed against rather than discovered, so the verification carries the whole weight. The density field produced by the hashed search is compared against a brute force $O(N^2)$ host sum over identical positions, and the worst relative difference across four thousand particles is three parts in ten million, which is single precision rounding. A single dropped or double counted neighbour would move a density by several percent. A separate health check counts particles sharing a bucket with a foreign cell and confirms the collision rate is real but small, so the test is exercising the defence rather than a table so sparse that no collisions occur.

3.2 A diagnostic that would have perturbed the run it measured

The problem. The fused variant carries the leapfrog half step velocity, which is not the velocity belonging to the current positions. The energy diagnostic needs the synchronised one. The obvious implementation is to shift the velocity array forward by half a step, take the measurement, and shift it back.

Why that is wrong. In single precision, adding a quantity and then subtracting it does not return the original bits. A run that happened to take a diagnostic would therefore follow a very slightly different trajectory from one that did not, which breaks the bit identity guarantee the resume path claims and makes the energy curve partly a measurement of its own observer.

Fix. The half step correction is applied inside the reduction kernel, which reads the velocity and the stored acceleration and forms $v + a\,\Delta t/2$ on the fly without writing anything back.

Verification. A test runs eighty steps taking a full diagnostic after every step, and asserts the final state is bit for bit identical to the same run with no diagnostics at all.

3.3 A test that failed for a reason that was entirely my fault

Symptom. The SPH kernel unit test `KernelsAreContinuousAtTheSupportBoundary` failed on first run, while every other kernel test passed.

Root cause. The test asserted that each kernel evaluated just inside its support boundary was within 10^{-3} of zero, in absolute terms. These kernels carry a $1/h^6$ normalisation: at a smoothing radius of 0.1 the spiky gradient peaks around 1.4×10^5 and the viscosity Laplacian around 1.4×10^6. An absolute tolerance of 10^{-3} against quantities of that magnitude is not a statement about continuity, it is a statement about nothing. The implementation was correct throughout.

Fix. The test now compares each kernel at the boundary against its own peak value, which is what continuity actually means for a quantity with a scale, and separately asserts the kernels are exactly zero outside the support so the transition is genuinely continuous rather than merely small.

Lesson. A tolerance without a scale is not a tolerance. Every numerical gate in this project is now expressed relative to something: the magnitude of the quantity, the machine epsilon, or a bound derived from the arithmetic.

3.4 A benchmark that recorded a random seed as a tile width

Symptom. The tile width figure had four bars, one of which sat at a tile width of `20260802`. That number is the random seed.

Root cause. The tile sweep opened the output file twice. It created a truncating output stream to write the header, and then called the row appending helper, which opens the same path independently in append mode. The header stream stayed open and buffered for the duration of the sweep and flushed at offset zero when it was finally destroyed, overwriting the beginning of the file with the header text. What survived was a file whose first rows were partly overwritten, so the column boundaries shifted and fields were read from the wrong columns.

Fix. The header is written through the same append path as the rows, with the file removed first, so only one handle to the file exists at any time.

Why it is in this report. Nothing failed. The benchmark ran, the CSV parsed, the plot rendered, and the number was wrong. It was caught by looking at the figure and asking what a tile width of twenty million meant. The general form of this defect is that two independent handles to one file will corrupt it, and buffering makes the corruption appear far from its cause.

3.5 LaTeX macro names cannot contain digits

Symptom. The report failed to compile with `Missing \begin{document}` pointing at a line in the generated macro file that looked entirely reasonable.

Root cause. A LaTeX control sequence is made of letters only. The offending definition was

```
\newcommand{\Poly6NormalisationError}{...}
```

in which the name ends at the digit, so this defined a macro called `Poly` followed by the literal text `6NormalisationError`. Literal text in the preamble is what produces that error message, and it is reported at the point of the text rather than at the mistake.

Fix. The three offending macros were renamed to spell their digits, and the generator now rejects any macro name that is not purely alphabetic, so this class of mistake fails at generation time with a message that names the problem rather than at compile time with one that does not.

3.6 BibTeX put en dashes back after they had been removed

Symptom. The dash gate failed on the compiled PDF, reporting seven en dashes in the bibliography, after the source bibliography had been cleaned of double hyphens and the gate had reported the sources clean.

Root cause. The bibliography style is what dashifies page ranges. The `plain` style passes page fields through a function that converts any run of hyphens into a TeX en dash ligature when it writes the intermediate bibliography file. Cleaning the source had no effect because the source was never the problem.

Fix. Page ranges are written with the word `to`, which the style passes through untouched. This also makes the bibliography consistent with the prose, which is required to write ranges the same way.

Why this justifies the whole design of the gate. The rule forbids a character in the finished document, so the gate has to check the finished document. A source only check would have reported clean while every reference in the printed bibliography broke the rule. Extracting the text of the compiled PDF and scanning that is what caught it, and it is the reason the gate runs last in the report build rather than first.

Chapter 4

What the failures had in common

4.1 The dangerous defects were the silent ones

Of the eleven entries in this report, three crashed, three failed to compile, and five produced plausible output while being wrong. The five silent ones took almost all the time.

A build targeting Turing instead of Blackwell compiles, links and runs. A benchmark whose columns have shifted parses cleanly and plots. An optimisation that relieves a resource nothing was waiting on delivers a small positive number that is easy to accept. A neighbour search that counts some particles twice produces a fluid that still behaves like a fluid. A diagnostic that perturbs the state it measures produces a smooth curve.

None of these announces itself, and none is found by testing that the program runs. Each was caught by a different mechanism, and the pattern is that each mechanism compares the thing against something independent: the object file against the intended architecture, the figure against what a tile width could plausibly be, the optimisation against a bound computed from measured bandwidth, the neighbour search against a brute force sum, the diagnostic run against an undiagnosed one.

4.2 A tolerance without a scale is not a tolerance

Two defects came from this directly. The kernel continuity test compared a quantity of order 10^{6} against an absolute tolerance of 10^{-3} and failed a correct implementation. Had the comparison gone the other way, against a quantity of order 10^{-6}, it would have passed anything at all.

Every gate in the project is now expressed against a scale that means something. The momentum gate is bounded by $\epsilon_{\mathrm{f32}} \sqrt{N} n_{\mathrm{steps}}$, which is what single precision arithmetic can explain, and it passes with six orders of magnitude to spare, so the number reported is informative rather than merely a pass. The force gate is relative to the magnitude of the acceleration being checked. The density gate is relative to rest density.

4.3 Size the optimisation before writing it

The fusion result is the clearest lesson in the project. The saving was computable in advance from two numbers already measured at Phase 0: the bytes removed and the achievable bandwidth. That calculation gives 0.019 percent as an absolute ceiling at 200,000 particles and takes two minutes.

The same calculation would have reframed the tiling work correctly as well. Tiling is worth a factor of 1.93, which no bandwidth argument can explain, because bandwidth was never the constraint. Knowing that in advance would have pointed at the load store pipeline from the beginning instead of at the memory bus.

4.4 Verification has to be independent of the thing verified

The strongest checks in this project are the ones where the reference was produced by a genuinely different route. The double precision host force implementation shares no code with the kernels and is written as literally as the force law can be written. The brute force density sum shares no code with the neighbour search. The chi squared test is checked by feeding it a deliberately wrong model that it must reject, so a test that could never fail would itself fail.

Where sharing is unavoidable it is deliberate and in the safe direction: the smoothing kernels and the force law are compiled from one header into both the device kernels and the host tests, so the two cannot drift apart. That is sharing an implementation to guarantee agreement, not sharing an assumption to guarantee a pass.

4.5 The environment is part of the experiment

Four of the eleven entries were toolchain problems, and one of them would have invalidated every performance number in the main report while leaving no trace except numbers that were slightly disappointing. Phase 0 of this build existed to probe the environment and record what it found, and that turned out to be time well spent rather than ceremony. The measured bandwidth figure recorded there is the denominator of the fusion ceiling calculation that explains the central performance result of the project.